

7.3. JSON (JavaScript Object Notation)

Es un formato ligero especialmente popular para el intercambio de datos entre aplicaciones.

Es totalmente compatible de forma nativa con JavaScript y como su propio nombre indica, **se basa en la sintaxis que tiene JavaScript para crear objetos.**

```
{  
  "name": "Richard",  
  "age": "33",  
  "hobbies": ["Biking", "Gaming", "Squash"],  
  "city": "Port Land",  
  "friends": [  
    {"name": "Chris", "age": 23, "city": "New York"},  
    {"name": "Emily", "age": 19, "city": "Atlanta"},  
    {"name": "Joe", "age": 32, "city": "New York"},  
    {"name": "Kevin", "age": 19, "city": "Atlanta"},  
    {"name": "Michelle", "age": 27, "city": "Los Angeles"},  
    {"name": "Robert", "age": 45, "city": "Manhattan"},  
    {"name": "Sarah", "age": 31, "city": "New York"}  
  ]  
}
```

7.3.1. Formato

Si comparamos un JSON con un objeto JavaScript, aparecen algunas ligeras diferencias y matices:

- Las propiedades del objeto deben estar entrecomilladas con «comillas dobles»
- Los textos deben estar entrecomillados con «comillas dobles»
- Sólo se puede almacenar tipos como textos, números, objetos, arrays, booleanos o null.
- Tipos de datos como function, date, expresiones regulares u otros, no es posible almacenarlos en un JSON.

7.3.2. Métodos

JavaScript dispone de métodos que facilitan la conversión entre cadenas JSON y objetos JavaScript:

- **JSON.parse(str):** Convierte una cadena de texto JSON válida en un objeto JavaScript.

```
const textoJSON = '{"nombre":"Carlos","edad":30,"ciudad":"Madrid"}';
```

```
const objeto = JSON.parse(textoJSON);
```

```
console.log(objeto.nombre); // Carlos
```

```
console.log(objeto.edad);   // 30
```

```
console.log(objeto.ciudad); // Madrid
```

- **JSON.stringify(obj):** Convierte un objeto JavaScript en una cadena JSON.

```
const persona = {  
  nombre: "Carlos",  
  edad: 30,  
  ciudad: "Madrid"  
};  
  
const json = JSON.stringify(persona);  
  
console.log(json);  
// {"nombre":"Carlos","edad":30,"ciudad":"Madrid"}
```

- **JSON.stringify(obj, props):** Igual que el anterior, pero filtra y mantiene solo las propiedades especificadas.

```
const usuario = {  
  nombre: "María",  
  edad: 28,  
  email: "maria@gmail.com",  
  password: "123456",  
  rol: "admin"  
};
```

// Solo queremos convertir estas propiedades a JSON

```
const props = ["nombre", "email", "rol"];  
  
const json = JSON.stringify(usuario, props);  
  
console.log(json);
```

```
{"nombre":"María","email":"maria@gmail.com","rol":"admin"}
```

- **JSON.stringify(obj, props, spaces):** Igual que el anterior, pero indenta el JSON con el número de espacios especificados.

```
const usuario = {
  nombre: "Carlos",
  edad: 35,
  email: "carlos@gmail.com",
  password: "abc123",
  rol: "admin"
};

// Propiedades que queremos incluir
const props = ["nombre", "email", "rol"];

// Espacios para formato legible
const spaces = 2;

const json = JSON.stringify(usuario, props, spaces);

console.log(json);
```

```
{
  "nombre": "Carlos",
  "email": "carlos@gmail.com",
  "rol": "admin"
}
```

Referencias:

<https://www.json.org/json-en.html>

<https://json5.org/>



7.3.3. Su historia con AJAX

AJAX, Asynchronous Javascript And XML, se utiliza para hacer peticiones asíncronas al servidor desde JavaScript.

- Mientras se espera la respuesta del servidor, la página continúa ejecutando su código y permitiendo la interacción con el usuario.
- Una vez que llega la respuesta del servidor, se ejecuta la función correspondiente.

Originalmente se utilizaba **XML** para el intercambio de información entre el servidor y el cliente, actualmente el formato más común es **JSON**, que es más simple y legible.

Métodos HTTP

Las peticiones Ajax usan el protocolo HTTP.

Hay diferentes tipos de petición:

- **GET:** para obtener datos sin modificar nada (SELECT en SQL). Se puede enviar datos en la URL (formato URLEncoded). Ej.: localhost/users/3



```
const xhr = new XMLHttpRequest();
```

```
xhr.open("GET", "https://jsonplaceholder.typicode.com/users");
```

```
xhr.send();
```

```
xhr.onload = function () {
```

```
  if (xhr.status === 200) {
```

```
    const datos = JSON.parse(xhr.responseText);
```

```
    console.log(datos);
```

```
  } else {
```

```
    console.error("Error:", xhr.status);
```

```
  }
```

```
};
```

- **POST:** para añadir un dato en el servidor (INSERT en SQL). Los datos van en el cuerpo de la petición HTTP.

```
const xhr = new XMLHttpRequest();

xhr.open("POST", "https://jsonplaceholder.typicode.com/users");
xhr.setRequestHeader("Content-Type", "application/json");

const usuario = {
  name: "Carlos",
  email: "carlos@gmail.com"
};

xhr.send(JSON.stringify(usuario));

xhr.onload = function () {
  if (xhr.status === 201 || xhr.status === 200) {
    console.log("Respuesta:", xhr.responseText);
  }
};
```

- **PUT**: similar a POST para actualizar datos del servidor (UPDATE en SQL). Los datos van en el cuerpo de la petición HTTP y la información para identificar el objeto a modificar en la url. El servidor sustituirá el objeto actual por el que se le pasa como parámetro.

```
const xhr = new XMLHttpRequest();
xhr.open("PUT", "https://jsonplaceholder.typicode.com/users/1");
xhr.setRequestHeader("Content-Type", "application/json");

const usuarioCompleto = {
  nombre: "Carlos López",
  email: "carlosnuevo@gmail.com",
  edad: 31
};

xhr.send(JSON.stringify(usuarioCompleto));
```

- **PATCH:** similar a PUT; pero mientras en PUT se pasan todos los campos del objeto, en PATCH sólo se pasan los campos que se quieren cambiar, permaneciendo igual el resto.

```
const xhr = new XMLHttpRequest();
xhr.open("PATCH", "https://jsonplaceholder.typicode.com/users/1");
xhr.setRequestHeader("Content-Type", "application/json");

const cambioParcial = {
  email: "carlosactualizado@gmail.com"
};

xhr.send(JSON.stringify(cambioParcial));
```

- **DELETE**: para eliminar un dato del servidor (DELETE en SQL). Se envía en URL, como GET.

```
const xhr = new XMLHttpRequest();  
xhr.open("DELETE", "https://jsonplaceholder.typicode.com/users/1");  
xhr.send();
```

Métodos HTTP – códigos de estado

El servidor acepta la petición, la procesa y envía una respuesta al cliente con el recurso solicitado, acompañada de cabeceras (tipo de contenido, idioma, etc.) y un código de estado.

Los códigos de estado más comunes son:

- **2xx**: Peticiones procesadas **correctamente**. Ejemplos: 200 (OK), 201 (Created, para una petición POST exitosa).

`GET /productos → 200 OK`

`POST /usuarios → 201 Created`

- **3xx: Redirección.** Ejemplos: 301 (El recurso se ha movido permanentemente a otra URL), 304 (El recurso no ha cambiado desde la última petición, se puede usar la caché).

301 Moved Permanently

`http://sitio.com → https://www.sitio.com`

304 Not Modified

No se vuelve a descargar la imagen o el CSS

- **4xx: Error del cliente.** Ejemplos: 404 (Not Found, recurso no encontrado), 401 (Not Authorized, acceso no autorizado).

401 Unauthorized

Se intenta acceder a una API sin enviar credenciales válidas.

404 Not Found

Se solicita una página o recurso que no existe.

- **5xx: Error del servidor.** Ejemplos: 500 (Error interno del servidor), 504 (Timeout, el servidor no responde).

500 Internal Server Error

Un error en el código del servidor (por ejemplo, una excepción no controlada).

504 Gateway Timeout

El servidor no recibe respuesta a tiempo de otro servicio del que depende.

Ejemplo de peticiones con métodos HTTP

Como hemos comentado, la información enviada por el **servidor al cliente** normalmente serán datos en **formato JSON** o XML (cada vez menos usado) que el cliente procesará y mostrará en la página al usuario.

Las peticiones Ajax se hacen a un servidor que proporcione una API.

→ Vamos a hacer peticiones sobre la API **JSONPlaceholder**:

<https://jsonplaceholder.typicode.com/todos>

... que tan solo tiene una lista de tareas en formato JSON



jsonplaceholder.typicode.com/todos

JSON

Datos sin procesar

Cabeceras

Guardar

Copiar

Contraer todo

Expandir todo



Filtrar JSON

▶ [0...99]:

▶ [100...199]:

▶ [0...99]:

▼ [100...199]:

- ▶ 100: { userId: 6, id: 101, title: "explicabo enim cumque porro aperiam occaecati minima", ... }
- ▶ 101: { userId: 6, id: 102, title: "sed ab consequatur", ... }
- ▶ 102: { userId: 6, id: 103, title: "non sunt delectus illo nulla tenetur enim omnis", ... }
- ▶ 103: { userId: 6, id: 104, title: "excepturi non laudantium quo", ... }
- ▶ 104: { userId: 6, id: 105, title: "totam quia dolorem et illum repellat voluptas optio", ... }
- ▶ 105: { userId: 6, id: 106, title: "ad illo quis voluptatem temporibus", ... }
- ▶ 106: { userId: 6, id: 107, title: "praesentium facilis omnis laudantium fugit ad iusto nihil nesciunt", ... }
- ▶ 107: { userId: 6, id: 108, title: "a eos eaque nihil et exercitationem incidunt delectus", ... }
- ▶ 108: { userId: 6, id: 109, title: "autem temporibus harum quisquam in culpa", ... }
- ▶ 109: { userId: 6, id: 110, title: "aut aut ea corporis", ... }
- ▶ 110: { userId: 6, id: 111, title: "magni accusantium labore et id quis provident", ... }
- ▶ 111: { userId: 6, id: 112, title: "consectetur impedit quisquam qui deserunt non rerum consequuntur eius", ... }

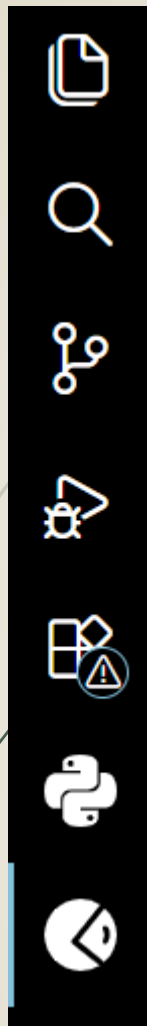
Es una **API REST** de prueba, llamada JSONPlaceholder.

- No es una página web normal
- No tiene interfaz gráfica
- Está pensada solo para practicar peticiones HTTP
- Tan sólo devuelve datos en formato JSON

API REST → es un servicio que expone recursos mediante URLs y permite operar sobre ellos usando métodos HTTP, devolviendo normalmente datos en formato JSON.

¿Cómo la usaremos?

Con herramientas como Postman, **extensión Postman desde VSCode** o con código (Java, JavaScript, Python, ...)



EXTENSIONS: ...

postman

Postman 2.1M
Streamline API develop...
Postman Install

Postman Ru... 104K
Run Postman tests dire...
mínguez Coloma Install

Live PostMan 22K
Launch a development...
wycliffe Pepela Install

vsc-postman 20K

Extension: Postman



Postman
Postman postman.com | 2,199,521
Streamline API development and testing v...
Install ☒ Auto Update

DETAILS FEATURES CHANGELOG

The Postman VS Code extension

Default workspace

New HTTP Requ...

+ Filter

GET  

<https://jsonplaceholder.typicode.com/todos>

✓ GET
POST
PUT
PATCH
DELETE
HEAD
OPTIONS
COPY
LINK

Send

Obtener todas las tareas ...

GET 

Send

Body Cookies Headers (26) Test Results

 Status: 200 OK Time: 102 ms Size: 5.04 KB

Pretty

Raw

Preview

JSON 



```
1  [  
2    {  
3      "userId": 1,  
4      "id": 1,  
5      "title": "delectus aut autem",  
6      "completed": false  
7    },  
8    {
```


Obtener la tarea con id 55 ...

GET  <https://jsonplaceholder.typicode.com/todos?id=55> **Send**

Params  Authorization Headers (7) Body Scripts Settings

Query Params

	Key	Value
☒	id	55

Body Cookies Headers (24) Test Results  Status: 200 OK Time: 457 ms Size: 1.26 KB

Pretty Raw Preview JSON  

```
1  [
2    {
3      "userId": 3,
4      "id": 55,
5      "title": "voluptatum omnis minima qui occaecati provident nulla voluptatem ratione",
6      "completed": true
7    }
8  ]
```

Modificar con PATCH la tarea con id 55 ...

PATCH

▼

https://jsonplaceholder.typicode.com/todos/55

Send

Params

Authorization

Headers (9)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

```
1 {  
2   ... "completed": false  
3 }
```

Body

Cookies

Headers (24)

Test Results

🌐 Status: 200 OK Time: 202 ms Size: 1.24 KB

Pretty

Raw

Preview

JSON ▼

↺

```
1 {  
2   "userId": 3,  
3   "id": 55,  
4   "title": "voluptatum omnis minima qui occaecati provident nulla voluptatem ratione",  
5   "completed": false  
6 }
```

7.3.4. API JSON Server

Es una API REST ficticia (mock) que tú ejecutas en tu propio ordenador, pero a diferencia de JSONPlaceholder, **SÍ** puedes crear, modificar y borrar datos de verdad mientras el servidor esté corriendo.

Es un backend o servidor API-REST que funciona bajo Node.js y que utiliza un fichero JSON como contenedor de los datos en lugar de una base de datos.

Es ideal para prototipado, desarrollo rápido y pruebas. Genera automáticamente las rutas CRUD (Create, Read, Update, Delete)....

Ejemplo JSON Server

→ Vamos a crear nuestra propia API

1. Instalamos JSON Server

```
npm install -g json-server
```

2. Creamos nuestro archivo de datos db.json

```
{
  "todos": [
    { "id": 1, "title": "Aprender APIs", "completed": false },
    { "id": 2, "title": "Usar Postman", "completed": true }
  ]
}
```

3. Arrancamos el servidor

```
json-server --watch db.json --port 3000
```

JSON Server started on PORT :3000

Press CTRL-C to stop

Watching db.json...

(..^..^..)

Index:

<http://localhost:3000/>

Static files:

Serving ./public directory if it exists

Endpoints:

<http://localhost:3000/todos>



localhost:3000/todos

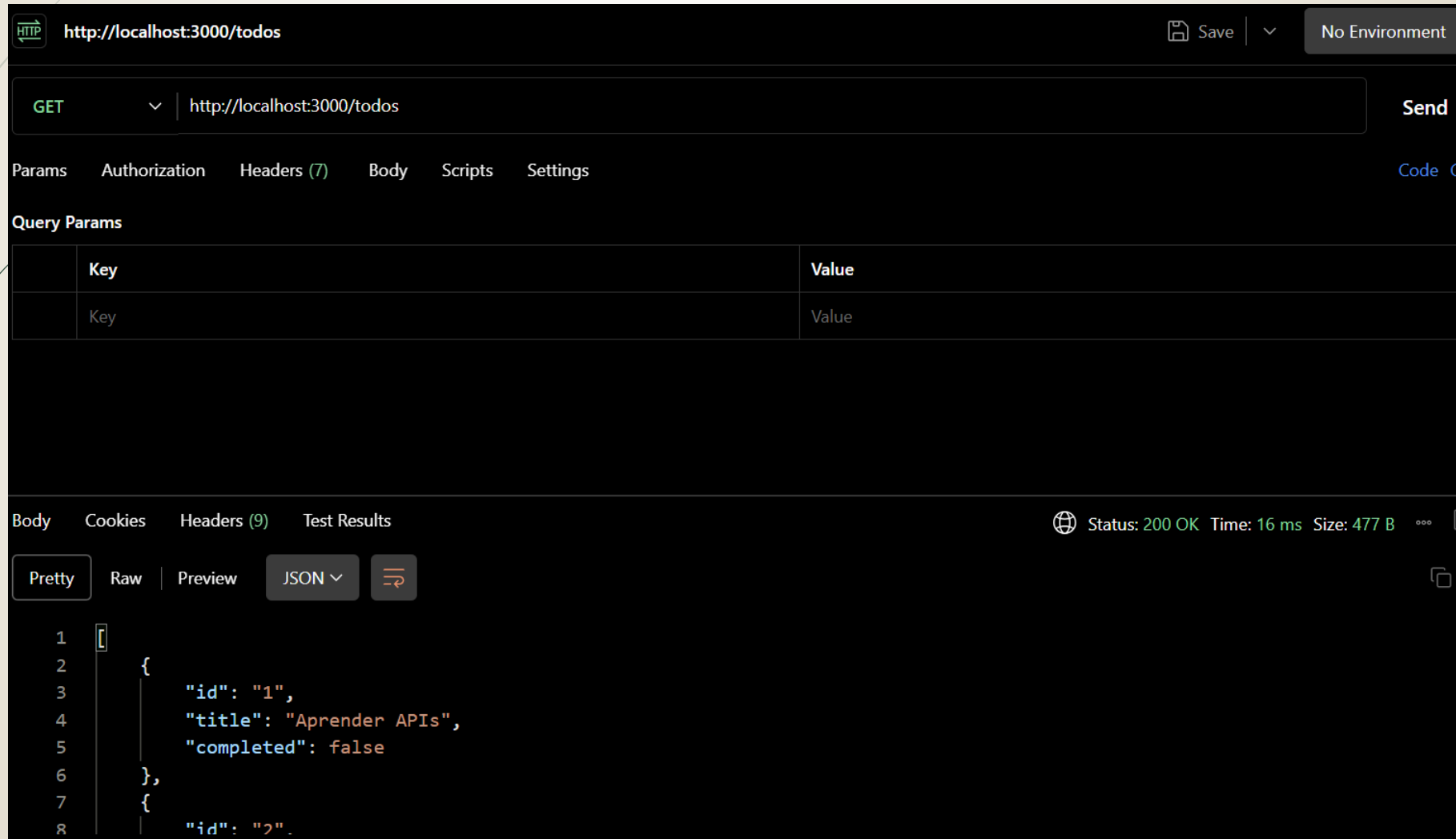
← → ↻ http://localhost:3000/todos

JSON Datos sin procesar Cabeceras

Guardar Copiar Contraer todo Expandir todo ⚙ Filtrar JSON

```
▼ 0:
  id:      "1"
  title:    "Aprender APIs"
  completed: false
▼ 1:
  id:      "2"
  title:    "Usar Postman"
  completed: true
```

4. Ya podremos realizar las peticiones HTTP, por ejemplo, con Postman en VSCode, tal como hemos hecho a JSONPlaceholder.



7.3.5. API Express

Express es un framework de aplicaciones web para Node.js.

Alternativa profesional a JSON Server para crear un backend o servidor real.

Es altamente flexible y se utiliza para construir aplicaciones web y APIs complejas.

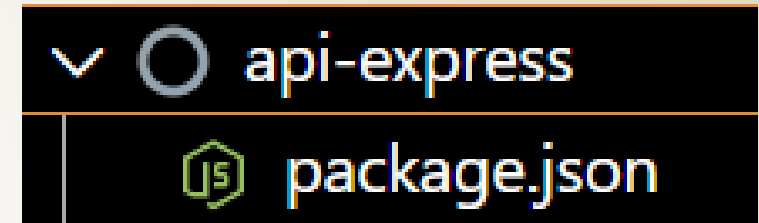
Adecuado para producción.

Ejemplo Express

→ Vamos a crear nuestra propia API

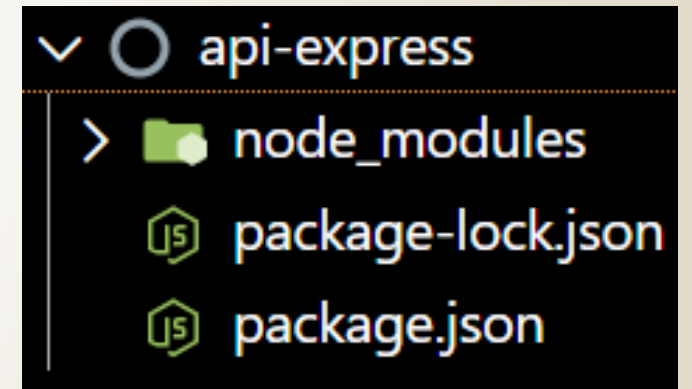
1. Creamos archivo **package.json**

```
npm init -y
```



2. Instalamos Express

```
npm install express
```



3. Creamos el archivo principal (**app.js** o index.js)

```
express > JS app.js > ...
```

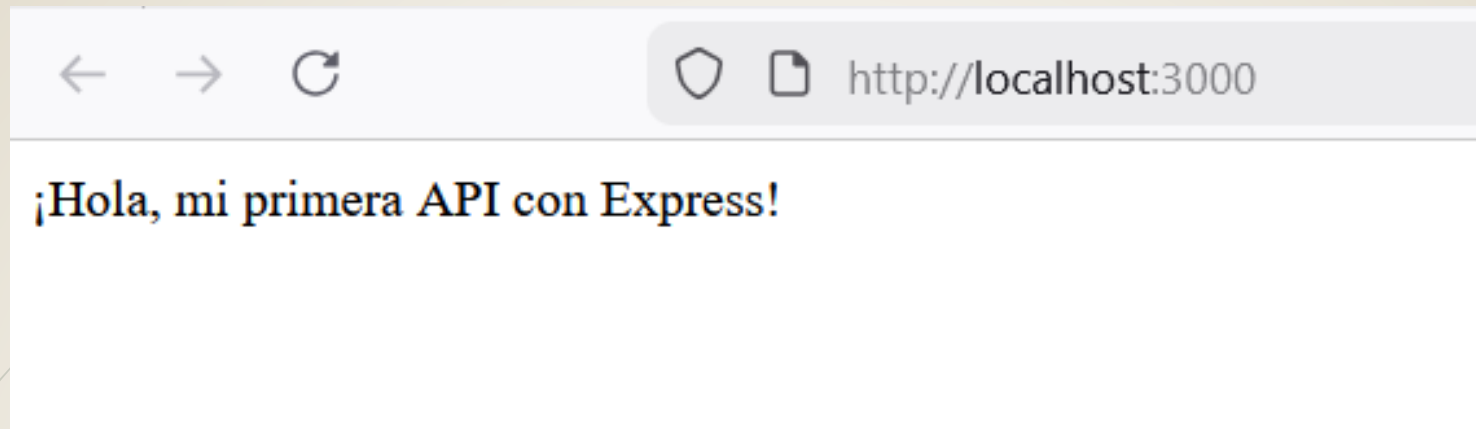
```
const express = require('express');  
const app = express();  
  
app.use(express.json()); // Para leer JSON  
  
app.get('/', (req, res) => {  
  res.send('¡Hola, mi primera API con Express!');  
});  
  
const PORT = 3000;  
app.listen(PORT, () => {  
  console.log(`Servidor corriendo en http://localhost:${PORT}`);  
});
```

```
✓ ○ api-express  
  > node_modules  
    JS app.js  
    package-lock.json  
    package.json
```

4. Ejecutamos la API.

```
node app.js
```

```
Servidor corriendo en http://localhost:3000
```



5. Definimos rutas (**endpoints**) de ejemplo en la API para recibir y enviar datos, agregando el siguiente código a **app.js**, y ya podremos hacer consultas a nuestra API, con Postman, por ejemplo.



```
// Obtener datos
app.get('/usuarios', (req, res) => {
  res.json([
    { id: 1, nombre: "Ana" },
    { id: 2, nombre: "Luis" }
  ]);
});

// Crear usuario
app.post('/usuarios', (req, res) => {
  const usuario = req.body;
  res.status(201).json({
    mensaje: "Usuario creado",
    usuario
  });
});
```

GET http://localhost:3000/usuarios

Send

Params Authorization Headers (7) Body Scripts Settings

Code

Query Params

	Key	Value
	Key	Value

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 17 ms Size: 285 B

Pretty Raw Preview JSON

```
1 [{"id":1,"nombre":"Ana"}, {"id":2,"nombre":"Luis"}]
```

Más información ...

<https://mauriciogc.medium.com/api-crud-node-js-y-express-260d194ed303>

<https://raul-alhena.medium.com/api-con-la-librer%C3%ADa-express-de-nodejs-8970f95cdfdc>

<https://expressjs.com/es/guide/routing.html>

<https://mauriciogc.medium.com/men-express-nodejs-mongodb-con-mongoose-4b3fdc1804cc>

<https://medium.com/@sergupe6/api-rest-con-nodejs-express-y-mongodb-286f3b14c717>

<https://github.com/urian121/CRUD-Nodejs-Express-MySQL/tree/master>